

10.2.2. Concise Implementation of LSTMs

As you might expect, by relying on the high-level APIs, we can implement LSTM concisely.

```
In [42]: use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip);
use AI::MXNet qw(mx nd);
use sml qw(show_plot);
IPerl->load_plugin('Chart::Plotly');
```

Subroutine main::mx redefined at /usr/local/share/perl5/5.30/AI/MXNet/NS.pm line 147.

10.2.3.1 Training data

```
In [2]: my $batch_size = 128;
#https://mxnet.apache.org/versions/1.7/api/python/docs/api/gluon/rnn/index.html
#https://mxnet.apache.org/versions/1.2.1/api/python/gluon/rnn.html
```

Out[2]: 128

```
In [3]: my $train_file_name = "../../../../../Datasets/VAD_data/201703211831/VAD_train1.csv";
my ($train_data, $train_header, $train_header_idx) = sml->load_csv($train_file_name);
my $row = $train_data->[0];
printf "%s\n", join " ", map { "$_-$train_header->[$_]" } 0 .. $#train_header;
printf "%s\n", join " ", map { "$_-$row->[$_]" } 0 .. $#row;
```

0-File_id 1-Time 2-Energy 3-zcr 4-intensity 5-F1_frequency 6-F1_bandwidth 7-F2_frequency 8-F2_bandwidth 9-c0 10-c1 11-c2 12-c3 13-c4 14-c5 15-c6 16-c7 17-c8 18-c9 19-c10 20-c11 21-c12 22-VAD
0-201703211831 1-40.08 2-0.0003439733585692011 3-50 4-0.012868976600981144 5-693.735396854883 6-478.61440947100306 7-3681.6107731055786 8-1671.0082996239516 9-1204.5889743284072 10-234.12557031372347 11-17.87708770041134 12--42.665291942714816 13--34.134639258051315 14--14.91188071702879 15--31.027709887692758 16--71.32573570814421 17--5.5203474105935655 18--74.38937382798042 19-52.75416915994289 20-2.8054111571452154 21--21.301539697239505 22-speech

Out[3]: 1

```
In [4]: # Converts last label into an integer
my $lookup = { pause => 0, speech => 1 };
($lookup, my $rlookup) = sml->str_column_to_int($train_data, -1, lookup => $lookup);
printf "lookup: %s\n", dump $lookup;
```

lookup: { pause => 0, speech => 1 }

Out[4]: 1

```
In [5]: # Separates the metadata columns
my $metadata_train = [ map { [ @{$_}[0 .. 1] ] } @{$train_data} ];
my $train = [ map { [ @{$_}[2 .. 22] ] } @{$train_data} ];
```

Out[5]: ARRAY(0x5566138a12d8)

```
In [6]: printf "metadata_train: %s\n", dump $metadata_train->[0];
$strain = nd->array($train);
nd->printf("Train: %.1f", $strain->slice(0));
```

metadata_train: [201703211831, 40.08]

Train: [0.0 50.0 0.0 693.7 478.6 3681.6 1671.0 1204.6 234.1 17.9 -42.7
-34.1 -14.9 -31.0 -71.3 -5.5 -74.4 52.8 2.8 -21.3 1.0]

<AI::MXNet::NDArray 21 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[6]: 1

```
In [7]: # Doing the same for the test data:
my $test_file_name = "../../../../../Datasets/VAD_data/202003021300/VAD_test1.csv";
my ($test_data, $test_header, $test_header_idx) = sml->load_csv($test_file_name);
sml->str_column_to_int($test_data, -1, lookup => $lookup);
# Separates the metadata columns
my $metadata_test = [ map { [ @{$_}[0 .. 1] ] } @{$test_data} ];
my $test = [ map { [ @{$_}[2 .. 22] ] } @{$test_data} ];
```

```
printf "metadata_test: %s\n", dump $metadata_test->[0];
$test = nd->array($test);
```

metadata_test: [202003021300, 43.56]

Out[7]: <AI::MXNet::NDArray 11115x21 @cpu(0)>

```
In [8]: my $X_train = $train->slice(':', [0, -1]);
my $X_test = $test->slice(':', [0, -1]);
printf "train: %s\n", $X_train;
printf "test: %s\n", $X_test;
```

train: <AI::MXNet::NDArray 12379x20 @cpu(0)>

test: <AI::MXNet::NDArray 11115x20 @cpu(0)>

Out[8]: 1

```
In [9]: nd->printf("Train:\n%.2f", $X_train->slice(0));
nd->printf("Test:\n%.2f", $X_test->slice(0));
```

Train:

```
[ 0.00  50.00  0.01 693.74 478.61 3681.61 1671.01 1204.59 234.13 17.88 -4
2.67 -34.13 -14.91 -31.03 -71.33 -5.52 -74.39 52.75 2.81 -21.30]
```

<AI::MXNet::NDArray 20 @cpu(0)> AI::MXNet::NDArray::Slice float32

Test:

```
[ 0.00  54.00  0.22 1469.96 839.25 2818.81 559.03 1455.12 162.88 -79.53 9
2.16 22.03 -83.45 -31.61 -52.89 -12.51 1.58 -44.82 31.20 12.37]
```

<AI::MXNet::NDArray 20 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[9]: 1

```
In [10]: # Standardize $X_train
my $all_data = nd->concat($X_train, $X_test, dim=>0);
my $X_means = nd->mean($all_data, axis=>0);
my $X_stdevs = nd->std($all_data, axis=>0, ddof=>1);
sml->standardize_dataset($X_train, $X_means, $X_stdevs);
nd->printf("Train:\n%.2f", $X_train->slice(0));
```

Train:

```
[ 0.06 -0.73 -0.35 -1.29 -0.64 0.98 0.89 0.67 1.05 0.26 -0.63 -0.91 0.77 0.01 -1.46 0.80
-2.22 2.86 0.61 -0.64]
```

<AI::MXNet::NDArray 20 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[10]: 1

```
In [11]: # Standardize $X_test
sml->standardize_dataset($X_test, $X_means, $X_stdevs);
nd->printf("Test:\n%.2f", $X_test->slice(0));
```

Test:

```
[ 0.63 -0.71 4.77 -0.37 -0.19 0.54 -0.14 1.57 0.41 -1.35 2.15 0.59 -1.22 -0.01 -0.76 0.51
0.91 -1.76 1.83 1.40]
```

<AI::MXNet::NDArray 20 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[11]: 1

```
In [12]: my $y_train = $train->slice(':', -1)->astype('int8');
my $y_test = $test->slice(':', -1)->astype('int8');
printf "train: %s\n", $y_train;
printf "test: %s\n", $y_test;
```

train: <AI::MXNet::NDArray 12379 @cpu(0)>

test: <AI::MXNet::NDArray 11115 @cpu(0)>

Out[12]: 1

Preparing data iterators with NTC format

Layout (str, default 'NTC') is the format of input and output tensors. N, T and C stand for batch size, sequence length, and feature dimensions respectively. Notice the data has 20 columns which has to be made bidimensional: sequence length and feature dimensions. A way of making these columns bidimensional it is taking 5 as sequence length and 4 as feature dimensions ($5 * 4 = 20$).

```
In [13]: # sub make_sequences {
# my ($X, $y, $seq_len) = @_;
#
# my $N = $X->shape->[0];
```

```

# my $base = nd->arange($N-$seq_len+1)->reshape([-1, 1]);
# my $offset = nd->arange($seq_len)->reshape([1, -1]);
# my $idx = $base + $offset;
# my $X_seq = nd->take($X, $idx);
# my $y_seq = nd->take($y, $idx);
#
# return ($X_seq, $y_seq);
# }

sub make_sequences {
    my ($X, $y, $seq_len) = @_;

    my $N      = $X->len;
    my $base   = nd->arange($N-$seq_len+1)->reshape([-1, 1]);
    my $offset = nd->arange($seq_len)->reshape([1, -1]);
    my $idx    = $base + $offset;
    my $X_seq  = nd->take($X,$idx);
    my $y_seq  = $y->slice([$seq_len-1, $N]);

    return ($X_seq, $y_seq);
}

```

```

In [14]: my $seq_len = 5;
my ($X_train_seq, $y_train_seq) = make_sequences($X_train, $y_train, $seq_len);
my ($X_test_seq, $y_test_seq)   = make_sequences($X_test, $y_test, $seq_len);

```

```

Out[14]: <AI::MXNet::NDArray 11111x5x20 @cpu(0)><AI::MXNet::NDArray 11111 @cpu(0)>

```

```

In [15]: nd->printf("X_train_seq->shape: %s", $X_train_seq->shape);
nd->printf("y_train_seq->shape: %s", $y_train_seq->shape);
nd->printf("Train:\n%.2f", $X_train_seq->slice([0, 2]));

```

```

X_train_seq->shape: [12375, 5, 20]
y_train_seq->shape: [12375]
Train:
[[[ 0.06 -0.73 -0.35 -1.29 -0.64  0.98  0.89  0.67  1.05  0.26 -0.63 -0.91  0.77  0.01 -1.46  0.
80 -2.22  2.86  0.61 -0.64]
 [ 0.69 -0.73 -0.28 -0.08  0.05 -0.90 -0.66  0.39  1.29 -0.60  0.14  0.10  0.38 -1.08 -0.80 -0.
11 -0.98  1.17  0.67  0.71]
 [ 0.59 -0.76 -0.09  0.23 -0.87 -0.90 -0.66  0.78  0.94 -1.18  1.21  0.22 -1.79 -1.53 -0.70  2.
16 -2.08  1.12 -0.19  2.20]
 [ 1.31 -0.71 -0.09 -1.18  0.03  0.31 -0.37  0.73  1.02 -1.21  0.01  0.55 -1.28  0.27  0.06 -1.
15 1.53  1.50  0.08 -1.10]
 [ 0.12 -0.70 -0.20 -1.09 -0.86  0.27 -0.21  0.54  1.75 -1.18 -0.82 -0.02 -0.38 -0.03  1.89 -1.
37 -1.29  3.10 -0.07 -1.00]]

[[[ 0.69 -0.73 -0.28 -0.08  0.05 -0.90 -0.66  0.39  1.29 -0.60  0.14  0.10  0.38 -1.08 -0.80 -0.
11 -0.98  1.17  0.67  0.71]
 [ 0.59 -0.76 -0.09  0.23 -0.87 -0.90 -0.66  0.78  0.94 -1.18  1.21  0.22 -1.79 -1.53 -0.70  2.
16 -2.08  1.12 -0.19  2.20]
 [ 1.31 -0.71 -0.09 -1.18  0.03  0.31 -0.37  0.73  1.02 -1.21  0.01  0.55 -1.28  0.27  0.06 -1.
15 1.53  1.50  0.08 -1.10]
 [ 0.12 -0.70 -0.20 -1.09 -0.86  0.27 -0.21  0.54  1.75 -1.18 -0.82 -0.02 -0.38 -0.03  1.89 -1.
37 -1.29  3.10 -0.07 -1.00]
 [ 0.40 -0.62 -0.17 -0.78 -0.50  1.32  2.09 -0.11  0.43  0.47 -0.56  0.32  0.90  0.97  0.08 -0.
29 -0.12  2.71 -0.77 -0.20]]]
<AI::MXNet::NDArray 2x5x20 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

```

Out[15]: 1

```

```

In [16]: sub load_array{ #@save
# Construct a Gluon data iterator.
my ($data_arrays, $batch_size, %args) = (splice (@_, 0, 2),
                                             is_train => 1,
                                             last_batch => 'keep', @_);

my ($X, $y) = @$data_arrays;
my $dataset = mx->gluon->data->ArrayDataset(data => $X,
                                             label => $y);

return mx->gluon->data->DataLoader($dataset,
                                   batch_size => $batch_size,
                                   shuffle    => $args{is_train},
                                   # 'keep', 'discard', 'rollover'
                                   last_batch => $args{last_batch} // 'discard');
}

```

10.2.3.2. Model

As compared with our concise implementation of softmax regression implementation (Section 3.7), the difference is that we add an LSTM layer. The first is our LSTM layer. The second is our output layer.

```
In [17]: package LSTM {
  use strict;
  use warnings;
  use Data::Dump qw(dump);
  use AI::MXNet qw(mx);
  use base ("AI::MXNet::Gluon::Block");

  sub new{
    my ($class, %args) = @_;

    my $self = $class->SUPER::new(%args);
    $$self->{model} = mx->gluon->nn->Sequential();
    $$self->{model}->add(mx->gluon->rnn->LSTM(hidden_size => $args{hidden_units},
    #                                     num_layers => $args{num_layer} // 1,
    #                                     layout => $args{layout} // 'NTC',
    #                                     dropout => $args{dropout} // 0,
    #                                     bidirectional => $args{bidirectional} // 0,
    #                                     input_size => $args{input_size} #C from NT'C'
    #                                     ));
    #
    $$self->{model}->add(mx->gluon->nn->Dense(units => $args{units},
    #                                     in_units => $args{in_units},
    #                                     flatten => 0,
    #                                     #activation => 'tanh'
    #                                     ));
    $$self->register_child($self->{model});

    $self->{lstm} = mx->gluon->rnn->LSTM(hidden_size => $args{hidden_units},
    #                                     num_layers => $args{num_layer} // 1,
    #                                     layout => $args{layout} // 'NTC',
    #                                     dropout => $args{dropout} // 0,
    #                                     bidirectional => $args{bidirectional} // 0,
    #                                     input_size => $args{input_size} #C from NT'C'
    #                                     );

    $self->{dense} = mx->gluon->nn->Dense(units => $args{units},
    #                                     in_units => $args{in_units},
    #                                     flatten => 0,
    #                                     #activation => 'tanh'
    #                                     );

    map{$self->register_child($self->{$_})} ('lstm', 'dense');

    return bless($self, $class);
  }

  #sub forward {
  #  my ($self, $X) = @_;
  #  return $self->{model}->forward($X);
  #}

  sub forward {
    my ($self, $X) = @_;

    my $H = $self->{lstm}->forward($X);

    # H : [batch_size, seq_len, num_hidden]

    $H = $H->slice(':', -1, ':')->sever; # materialized slice. -1 preserves just the last predi
    # H : [batch_size, num_hidden]

    return $self->{dense}->forward($H);
  }
}
```

```
1;
}
```

Out[17]: 1

```
In [18]: my $net = new LSTM(hidden_units => 8,
                           num_layer   => 1,
                           layout      => 'NTC',
                           dropout     => 0.2,
                           bidirectional => 0,
                           input_size  => $X_train_seq->shape->[-1], # Feature dimensions C from NTC'
                           units       => 2, # number of classes.
                           in_units    => 8); # matches with hidden_units. Double it if bidirectional

# print $net->{model};
print join "\n", map{$net->{$_}} ('lstm', 'dense');

LSTM(20 -> 8, NTC, dropout=0.2)
Dense(2 -> 8, linear)
```

Out[18]: 1

```
In [19]: my $train_iter = load_array([$X_train_seq, $y_train_seq], $batch_size, is_train => 1, last_batch
my $test_iter  = load_array([$X_test_seq, $y_test_seq], $batch_size, is_train => 0, last_batch

$net->collect_params->initialize(init => mx->init->Xavier(), force_reinit => 1);
#$net->collect_params->initialize(init => mx->init->Xavier(magnitude=>2), force_reinit => 1);
#$net->collect_params->initialize(init => mx->init->Normal(sigma => 0.01), force_reinit => 1);
```

10.2.3.3. Training

The training loop is exactly the same as when we implemented softmax regression. This modularity enables us to separate matters concerning the model architecture from orthogonal considerations.

```
In [20]: sub train_epoch_ch3{
  # Train a model within one epoch (defined in Chapter 3).
  # Sum of training loss, sum of training accuracy, no. of examples
  my ($net, $train_iter, $loss, $updater) = @_;

  my ($y_hat, $l, $X, $y);

  while ( my $batch = <$train_iter> ) {
    ($X, $y) = @$batch;
    # Computes gradients and updates parameters

    mx->autograd->record(sub {
      $y_hat = $net->($X);
      $l      = $loss->($y_hat, $y->astype('float32'));
    });
    $l->backward();

    $updater->step($X->len());
  }
}
```

```
In [21]: sub train_ch3{
  # Train a model (defined in Chapter 3).
  my ($net, $train_iter, $test_iter, $loss, $num_epochs, $updater) = @_;

  my (@train_metrics, $test_acc);
  for (my $epoch = 0; $epoch < $num_epochs; $epoch++){
    train_epoch_ch3($net, $train_iter, $loss, $updater);
  }
}
```

```
In [22]: my $lr    = 0.01;
my $loss = mx->gluon->loss->SoftmaxCrossEntropyLoss();
```

```
In [23]: my $trainer = mx->gluon->Trainer($net->collect_params(),
                                         optimizer => 'adam', # sgd
                                         optimizer_params=>{ learning_rate => $lr });#, clip_gradient =
```

```
Out[23]: AI::MXNet::Gluon::Trainer=HASH(0x556612f807c8)
```

```
In [24]: my ($train_model, $num_epochs, $animator, $model_file_name) =  
        (1, 10, undef, '09_02_02-Concise_Implementation_of_LSTM.mdl');
```

```
Out[24]: 11009_02_02-Concise_Implementation_of_LSTM.mdl
```

```
In [25]: if ($train_model){  
        train_ch3($net, $train_iter, $test_iter, $loss, $num_epochs, $trainer);  
        $net->save_parameters($model_file_name);  
    }else{  
        $net->load_parameters($model_file_name);  
    }
```

10.2.3.4. Prediction

```
In [26]: sub get_probs {  
        my ($net, $data_iter, $batch_size, $seq_len) = @_;  
  
        my $hidden_state = $net->begin_state($batch_size);  
        my (@logits, $y_hat, $X, $y);  
        while(my $batch = <$data_iter>){  
            ($X, $y) = @$batch;  
            ($y_hat, $hidden_state) = $net->forward($X, $hidden_state);  
            push @logits, $y_hat;  
        }  
  
        # Assumes the first predictions are repeated because seq_len just predicts the last element o  
        my $first_logits = nd->tile($logits[0]->slice(0), reps => $seq_len -1->reshape([$seq_len -1,  
        my $logits = nd->concat($first_logits, @logits, dim=>0);  
        return nd->softmax($logits, axis => 1);  
    }
```

```
In [27]: my $probs = get_probs($net, $test_iter, $batch_size, $seq_len);  
my $mask = ($probs + 0.1) > 0.6;  
my $preds = $mask->argmax(axis => 1)->astype('int8');  
my @preds = map {$rlookup->{$_}} $preds->tolist;  
print dump @preds[150 .. 200];
```

[illegible]

```
Out[27]: 1
```

```
In [30]: my $both = nd->concat($y_test->expand_dims(axis=>1), $preds->expand_dims(axis=>1), dim=>1);
print $both;
nd->printf("Actual\\tPredictions:\\n%s", $both->slice([30, 80]));
my $accuracy = sml->accuracy_metric($y_test, $preds);
printf 'Accuracy: %0.2f%%', $accuracy;
```

```
<AI::MXNet::NDArray 50x2 @cpu(0)> AI::MXNet::NDArray::Slice int8
Accuracy: 97.68%
```

```
Out[30]: 1
```

```
In [31]: my ($clases, $matrix) = sm1->confusion_matrix($y_test, $preds);
printf "clases: %s\n", $clases->asstr;
nd->printf("confusion matrix:\n%s\n", $matrix);
```

```
classes: [0 1]
confusion matrix:
[[1713   98]
 [ 160 9144]]
```

```
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[31]: 1
```

```
In [32]: # Test MAE
my $mae = sml->mae_metric($y_test, $preds);
printf "mae:%s\n", $mae;
```

```
mae:0.023
```

```
Out[32]: 1
```

```
In [33]: # Test RMSE
my $rmse = sml->rmse_metric($y_test, $preds);
printf "rmse:%s\n", $rmse;
```

rmse:0.000

Out[33]: 1

```
In [34]: my $pause_probs = $probs->slice(':', 0);
my $speech_probs = $probs->slice(':', 1);
```

Out[34]: <AI::MXNet::NDArray 11115 @cpu(0)>

```
In [35]: my $positive_class = 0;
my $positive_probs = $probs->slice(':', $positive_class);
```

Out[35]: <AI::MXNet::NDArray 11115 @cpu(0)>

```
In [36]: # Calculate TPR and FPR for a specific threshold
(my $fpr, my $tpr, $matrix) = sml->perf_metrics($y_test, $positive_probs, 0.5, positive_class=>

# Print sensitivity and specificity
printf "tpr: %s, 1 - fpr: %s\n", $tpr->asstr, (1 - $fpr)->asstr;
printf "confusion matrix:\n%s\n", $matrix->asstr;
```

tpr: [0.9459], 1 - fpr: [0.9828]
 confusion matrix:
 [[1713 98]
 [160 9144]]

Out[36]: 1

```
In [37]: # Calculate TPR and FPR for various decision thresholds
my $thresholds = nd->arange(101) / 100;
nd->printf("thresholds: %.2f\n", $thresholds->slice([0, 5]));
```

thresholds: [0.00 0.01 0.02 0.03 0.04]
 <AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[37]: 1

```
In [38]: # 1. Execute loop over thresholds - returning arrays of tensors
my ($fprs_array, $tprs_array) = (zip map {
    my ($f, $t) = sml->perf_metrics($y_test, $positive_probs, $_, positive_class=>$positive_cla
    [$f, $t];
} @$thresholds);

# 2. Stack the tensor pieces into continuous vectors
# (No string-to-number casting overhead!)
my $fprs = nd->concat(@$fprs_array, dim=>0);
my $tprs = nd->concat(@$tprs_array, dim=>0);

# 3. Print the results with your perfect numpy-style visual alignment
printf "fprs vector:\n%s\n", $fprs->slice([0, 10])->asstr;
printf "tprs vector:\n%s\n", $tprs->slice([0, 10])->asstr;
```

fprs vector:
 [1 0.1412 0.1093 0.0947 0.0845 0.0786 0.0731 0.0677 0.0632 0.0603]
 tprs vector:
 [1 0.9884 0.9845 0.9818 0.9807 0.9790 0.9779 0.9768 0.9763 0.9757]

Out[38]: 1

```
In [39]: # 1. Obtain the indices that sorts the FPR tensor in ascending order
my $sorted_indices = nd->argsort($fprs, axis=>0);

# 2. Sort both vectors using nd->take over the $sorted_indices
my $sorted_fprs = nd->take($fprs, $sorted_indices);
my $sorted_tprs = nd->take($tprs, $sorted_indices);

# 3. Visual verification of the sort of FPR aligned with TPR
printf "sorted fprs:\n%s\n", $sorted_fprs->slice([20, 30])->asstr;
printf "sorted tprs:\n%s\n", $sorted_tprs->slice([20, 30])->asstr;
```

```
sorted fprs:
[0.0054 0.0055 0.0056 0.0058 0.0059 0.0063 0.0069 0.0071 0.0074 0.0076]
sorted tprs:
[0.9221 0.9232 0.9249 0.9255 0.9271 0.9277 0.9288 0.9293 0.9321 0.9332]
```

Out[39]: 1

```
In [40]: # Calculate the area under the ROC curve (AUC) using the trapezoid rule
my $auc = sml->trapz($sorted_fprs, $sorted_tprs);
printf "Area under the ROC curve (AUC) for %s prediction: %0.2f\n", $rlookup->{$positive_class}
```

Area under the ROC curve (AUC) for pause prediction: 0.99

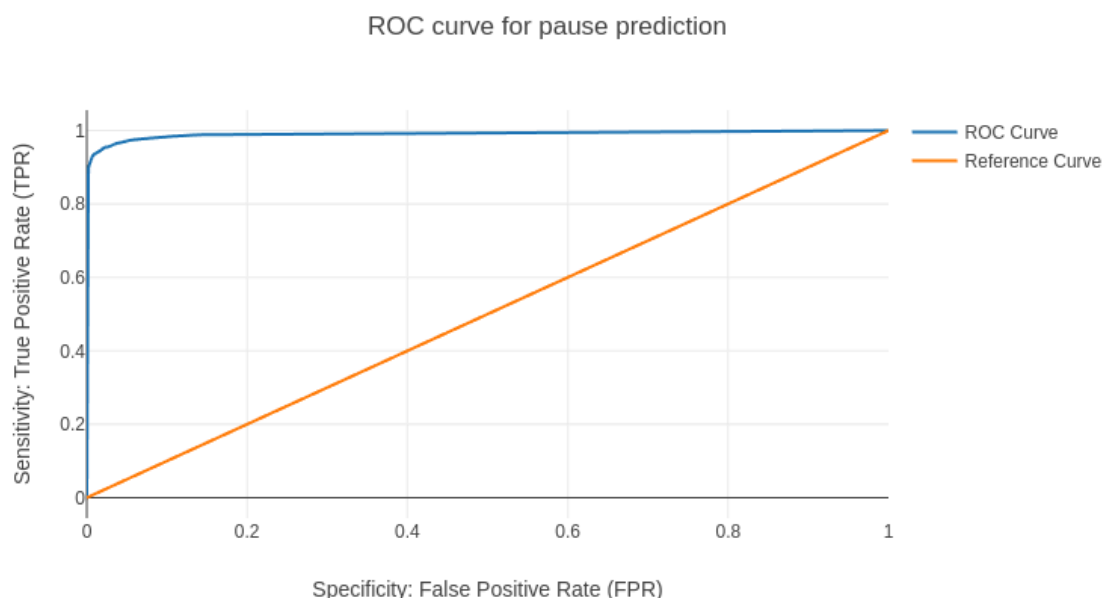
Out[40]: 1

```
In [43]: # Plot the ROC curve using Chart::Plotly
my $tracel = new Chart::Plotly::Trace::Scatter(
  x => $sorted_fprs->asperl,
  y => $sorted_tprs->asperl,
  mode => 'lines',
  name => 'ROC Curve'
);

my $trace2 = new Chart::Plotly::Trace::Scatter(
  x => [0, 1],
  y => [0, 1],
  mode => 'lines',
  name => 'Reference Curve'
);

my $plot = new Chart::Plotly::Plot(
  traces => [$tracel, $trace2],
  layout => {
    title => sprintf('ROC curve for %s prediction', $rlookup->{$positive_class}),
    xaxis => { title => 'Specificity: False Positive Rate (FPR)' },
    yaxis => { title => 'Sensitivity: True Positive Rate (TPR)' }
  }
);

# Show the graph directly in IPerl
IPerl->display($plot);
```



10.2.3.5 Summary

- Using high-level APIs, we can implement LSTM much more concisely.

- For the same classification problem, the implementation of an LSTM is the similar as that of softmax regression except for additional LSTM layer.
- Therefore, you have now reached the state of the art of the years 2000.